

Project on signal and image deconvolution

Charles Soussen, Elisabeth Lahalle

Image restoration : problematic

The problem considered is the reconstruction of a signal/image having undergone a low-pass linear filtering. The signal measured at the output of the filter is also assumed to be corrupted by additive uncertainties (quantization noise, measurement noise). The resolution of this problem involves a first step of modeling the relationship that links the measured signal to the signal to be reconstructed (direct problem). The second step consists in solving the inverse problem to reconstruct the input signal of the filter from the measured signal. We will first focus on solving an inverse problem in the case of 1D signals before generalizing to the 2D case.

1. Deconvolution of a piecewise constant signal

The first exercise aims to understand the unstable nature of the deconvolution operation, when it is done in a naive way either by division in the frequency domain (**deconv**) or by inverse filtering. It is strongly inspired by the example given in introduction of the book [J. Idier, 2008, *Bayesian approach for inverse problems*]. When dealing with noisy data, these deconvolution methods may be unusable even for very weak noise levels. This is an *ill-posed* inverse problem. The second exercise will illustrate the solution(s) obtained by linear regularization quadratic.

The problem considered is the reconstruction of a signal having undergone Gaussian filtering. The impulse response of a Gaussian filter is defined by :

$$g(t) = \frac{1}{\sigma_h \sqrt{2\pi}} \exp\left(-\frac{(t - m_h)^2}{2\sigma_h^2}\right). \quad (1)$$

The signal measured at filter output is corrupted by additive noise.

Discretization of the impulse response and simulation of the input signal

The sampling frequency is fixed $f_e = 1$ Hz. Signal samples are denoted $x[n]$, those of the impulse response of the filter $h[n] = g(n/f_e)$ and those of the signal measured in output $y[n]$. The signal measured at the output is subject to additive uncertainties noted $w[n]$. The direct problem can then be modeled as follows :

$$y[n] = (x \star h)[n] + w[n].$$

1. For simulation purpose, compute the impulse response $h[n]$ of the filter Gaussian obtained by discretization of equation (1). The filter is causal with parameters $m_h = 15$ and $\sigma_h = 5$. The impulse response is truncated for a time horizon of 30 s. Draw the impulse response.
2. The input signal is assumed piecewise constant :

$$x(t) = \begin{cases} 0 & \text{for } t \in [0; \frac{L_x}{2}] \\ 1 & \text{for } t \in [\frac{L_x}{2}; L_x] \end{cases}.$$

Simulate the corresponding discrete-time signal $x[n]$ for an observation time $L_x = 100$ s (note : it is recommended to use an instruction such as $\mathbf{x}=[\mathbf{x1} \ \mathbf{x2}]$, where $\mathbf{x1}$ and $\mathbf{x2}$ represent two successive sections of the signal).

1.1 Naive approach by inverse filtering

Part 1 : Simulation of the direct problem

1. Simulate the noiseless output signal $y_0[n] = (x \star h)[n]$ (function `conv`).
2. The SNR is defined by $\text{SNR} = 10 \log_{10}(P_{y_0}/P_w)$ where P_{y_0} and P_w are the powers of the signal and noise components. Simulate noise addition for a given SNR (SNR = 50, 30, 20 or 10 dB). Draw the noisy signal $y = y_0 + w$ on a figure.

Part 2 : Interpretation of inversion in the Fourier domain

1. Compute with Matlab the discrete Fourier transform (DFT) H of the filter. Trace its frequency response.
2. Implement deconvolution by inverse filtering in the frequency domain : compute the DFT X_{rec} from the DFTs H and Y , then compute the time representation x_{rec} . Draw the reconstructed signal x_{rec} (for example, for SNR = 50 dB).
3. To analyze the poor performance of naive inversion, draw :
 - the power spectral density (PSD) $S_w(f)$ of the noise ;
 - the PSD $S_y(f)$ of the output signal ;
 - the energy spectral density of the inverse filter.
4. Elaborate the frequency interpretation of the deconvolution problem.

1.2 Least squares approach

Now, the convolution $y_0[n] = (x \star h)[n]$ is simulated by the matrix equation $\mathbf{y}_0 = \mathbf{H}\mathbf{x}$ where :

$$\mathbf{H} = \begin{bmatrix} h[0] & 0 & \dots & 0 & 0 & \dots & \dots & 0 \\ h[1] & h[0] & \dots & \dots & 0 & 0 & \dots & 0 \\ \vdots & & \ddots & & & & & \\ h[N_h - 1] & \vdots & \dots & h[0] & 0 & \dots & \dots & 0 \\ 0 & h[N_h - 1] & \dots & \dots & \ddots & & \dots & 0 \\ \vdots & & & & \ddots & & \vdots & \\ 0 & 0 & \dots & \dots & \dots & \dots & \dots & 0 \\ 0 & 0 & \dots & 0 & \dots & \dots & \dots & \dots \end{bmatrix}.$$

Part 1 : Simulation of the direct problem by the matrix approach

The use of careful parameterization in matlab codes is recommended to allow easy modification of the parameters and display of results.

1. Determine dimension N_y of output vector $\mathbf{y}_0$ from dimensions N_x and N_h of input vectors $\mathbf{x}$ and impulse response $\mathbf{h}$ in order to obtain the same number of samples than with function `conv`.
2. Construct the convolution matrix $\mathbf{H}$ using function `toeplitz`. It is recommended to initialize the first row vector and the first column vector of $\mathbf{H}$ with null vectors of appropriate dimension using function `zeros`, then replace some of their elements with the elements of $\mathbf{h}$.

3. Simulate $\mathbf{y}_0$ by a matrix-vector product.

Part 2 : Deconvolution by regularized least squares

This part implements the the solution obtained by least squares from noisy data $\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{w}$.

1. Compute the deconvolved signal using the least squares estimator (minimization of $\|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2$ with respect to $\mathbf{x}$).
2. Plot this deconvolved signal and compute the reconstruction error energy.
3. Build the matrix $\mathbf{D}_1$ of dimension $(N_x - 1) \times N_x$ such that $\mathbf{D}_1\mathbf{x}$ yields the first order differences $x[n + 1] - x[n]$. You will again use function `toeplitz`.

$$\mathbf{D}_1 = \begin{bmatrix} 1 & -1 & 0 & 0 & \dots & \dots & 0 \\ 0 & 1 & -1 & 0 & \dots & \dots & 0 \\ \vdots & & \ddots & \ddots & & & \vdots \\ \vdots & & & \ddots & \ddots & & \vdots \\ 0 & \dots & \dots & 0 & 1 & -1 & 0 \\ 0 & \dots & \dots & 0 & 0 & 1 & -1 \end{bmatrix}.$$

4. Compute the regularized solution $\hat{\mathbf{x}}(\alpha)$, which minimizes $\mathcal{F}_\alpha(\mathbf{x}) = \|\mathbf{y} - \mathbf{H}\mathbf{x}\|^2 + \alpha \|\mathbf{D}_1\mathbf{x}\|^2$. You will determine the regularization coefficient α in an empirical way (for example, by testing several values in a large interval, like $[10^{-7}, 10^2]$).
5. Study the evolution of the optimal coefficient α with respect to the noise variance.
6. Draw the evolution of the condition number of matrix $\mathbf{H}^T\mathbf{H} + \alpha\mathbf{D}_1^T\mathbf{D}_1$ (related to the definition of $\hat{\mathbf{x}}(\alpha)$) as a function of α .

2. Circulante approximation and fast algorithms

We now consider the version of convolution product which yields a signal $y_0 = x \star h$ of same size as x , with $\mathbf{x} = (x[1], x[2], \dots, x[N_x])^T$ and $\mathbf{y}_0 = (y_0[N_h], \dots, y_0[N_h + N_x - 1])^T \in \mathbb{R}^{N_x}$. Matrix formulation $\mathbf{y}_0 = \mathbf{H}\mathbf{x}$ involves a matrix square $\mathbf{H}$ that is Toeplitz and upper triangular :

$$\mathbf{H} = \begin{bmatrix} h[N_h - 1] & h[N_h - 2] & \dots & h[0] & 0 & \dots & \dots & 0 \\ 0 & h[N_h - 1] & \dots & \dots & h[0] & 0 & \dots & 0 \\ \vdots & & \ddots & & & & & \\ 0 & \dots & 0 & h[N_h - 1] & \dots & \dots & \dots & h[0] \\ 0 & \dots & \dots & 0 & h[N_h - 1] & \dots & \dots & h[1] \\ \vdots & & & & & \ddots & & \vdots \\ 0 & 0 & \dots & \dots & \dots & 0 & h[N_h - 1] & h[N_h - 2] \\ 0 & 0 & \dots & \dots & \dots & \dots & 0 & h[N_h - 1] \end{bmatrix}.$$

In the following, we consider the same problem as before, that is, the reconstruction of the piecewise constant signal $\mathbf{x}$ from noisy data $\mathbf{y} = \mathbf{y}_0 + \mathbf{w}$.

2.1 Least squares

The least squares criterion is given by :

$$\mathcal{F}_0(\mathbf{x}) = (\mathbf{y} - \mathbf{H}\mathbf{x})^T(\mathbf{y} - \mathbf{H}\mathbf{x}).$$

Since $\mathbf{H}$ is square, the minimum of $\mathcal{F}_0$ is given by $\hat{\mathbf{x}} = \mathbf{H}^{-1}\mathbf{y}$.

Matrix $\mathbf{H}$ to be inverted is of size $N_x \times N_x$. For large N_x , direct inversion is numerically time consuming or even impossible. This said, the inverse of $\mathbf{H}$ can be fastly (approximately) computed using the properties of circulant matrices and fast Fourier transform calculations (`fft` function of matlab), see lecture slides. The algorithm is summarized in the box below :

1. Compute the FFT λ_h of the impulse response on N_x points.
2. Compute the FFT $\mathbf{Y}$ of the data.
3. Compute $\widehat{\mathbf{X}}$ as a elementwise division (denoted `./` under matlab) of vectors $\mathbf{Y}$ and λ_h .
4. Get $\hat{\mathbf{x}}$ by inverse FFT of $\widehat{\mathbf{X}}$.

It can be noted that this method does not require to compute nor store matrix $\mathbf{H}$.

Implement this algorithm, and observe that the method suffers from the same flaws as the naive deconvolution implemented in part 1.

2.2 Regularized least squares

The quadratic penalized criterion reads :

$$\begin{aligned} \mathcal{F}_\alpha(\mathbf{x}) &= (\mathbf{y} - \mathbf{H}\mathbf{x})^T(\mathbf{y} - \mathbf{H}\mathbf{x}) + \alpha \sum_{n=1}^{N_x-1} (x[n+1] - x[n])^2 \\ &= (\mathbf{y} - \mathbf{H}\mathbf{x})^T(\mathbf{y} - \mathbf{H}\mathbf{x}) + \alpha \mathbf{x}^T \mathbf{D}_1^T \mathbf{D}_1 \mathbf{x} \end{aligned}$$

with $\mathbf{D}_1$ the matrix of first order differences, of size $(N_x - 1) \times N_x$, defined in part 1. Criterion $\mathcal{F}_\alpha$ is quadratic. Its minimizer is given by :

$$\hat{\mathbf{x}} = (\mathbf{H}^T \mathbf{H} + \alpha \mathbf{D}_1^T \mathbf{D}_1)^{-1} \mathbf{H}^T \mathbf{y}. \quad (2)$$

Here again, we carry out a circulant approximation of $\mathbf{H}$ and $\mathbf{D}_1$ to speed up calculations (see lecture slides). The frequency response of the inverse filter reads :

$$\mathbf{g}_{RLS}[n] = \frac{(\lambda_h[n])^*}{|\lambda_h[n]|^2 + \alpha |\lambda_d[n]|^2} \quad \text{for } n = 1, 2, \dots, N_x. \quad (3)$$

The procedure to follow is given below.

1. Construct λ_h the FFT of the impulse response on N_x points.
2. Construct λ_d the FFT of $[1; -1]$ on N_x points.
3. Construct $\mathbf{Y}$ the FFT of the data.
4. Calculate $\widehat{\mathbf{X}}$ as the term-for-term product (`.*`) of the vectors $\mathbf{Y}$ and $\mathbf{g}_{RLS}$, with

$$\mathbf{g}_{RLS}[n] = \frac{(\lambda_h[n])^*}{|\lambda_h[n]|^2 + \alpha |\lambda_d[n]|^2} \quad \text{for } n = 1, 2, \dots, N_x.$$

5. Get $\hat{\mathbf{x}}$ by inverse FFT of $\widehat{\mathbf{X}}$.

3. Image deconvolution by quadratic regularization (ℓ_2)

We want to deblur an image, *i.e.*, to restore a image which sharp edges from a blurred version of it. One can imagine a photo taken with an imperfect focus. The image of a point is not a point but a stain. The measured image is then the superposition of the stains resulting from each point of the true image.

Knowing the point spread function (PSF) of the imaging device, the goal is to reconstruct the image with sharp edges from the blurred image.

3.1 Modeling the direct problem

The simplest model to describe such a transformation is a 2D convolution.

$$y[m, n] = \sum_{p=-P}^P \sum_{q=-Q}^Q h[p, q] x[m-p, n-q] + w[m, n]. \quad (4)$$

$x[m, n]$ refers to the actual image and $y[m, n]$ to the blurred data. In order to take into account measurement and modeling errors, we introduce a noise $w[m, n]$, assumed spatially white.

Note that the above filter is not causal since the $y[m, n]$ depends on pixel $x[m, n]$ and its neighbors in the four directions (up, down, left and right). As in the 1D case, it is fundamental to note that h is a low pass filter. Most measuring devices are of this type : they cannot « follow up too high frequencies ». It is then clear that the system eliminates the high frequency components of the image, which is why the inverse problem is particularly difficult. One aims to reconstruct the high frequency content which has disappeared from the data.

3.2 Image deconvolution

The work to be done consists in constructing an estimator $\hat{x}[m, n]$ of $x[m, n]$ from measurements $y[m, n]$. One can develop rigorous calculations analogous to the 1D case. However, these calculations are far more tedious because matrices have a more complex structure (Toeplitz block matrices with Toeplitz blocks). Moreover, the circulant approximation is harder to deal with. This is why the expression of inverse filters is given without proof. A few remarks :

- Images, impulse responses and first-order operators are two-dimensional, which is why FFTs are replaced by 2D FFTs.
- More precisely, if the image to be processed has N rows and N columns, all 2D FFTs must be taken over N rows and N columns.

The structures of the procedures generalizing the 1D case are given below, first in the case of least squares and then for regularized least squares.

1. Compute the 2D FFT λ_h of the point spread function on $N \times N$ points.
2. Compute the 2D FFT $\mathbf{Y}$ of data.
3. Compute $\widehat{\mathbf{X}}$ as a componentwise division (denoted $./$ in matlab) of vectors $\mathbf{Y}$ and λ_h .
4. Get $\hat{x}$ by inverse 2D FFT of $\widehat{\mathbf{X}}$.

1. Compute the 2D FFT λ_h of the PSF on $N \times N$ points.
2. Compute the 2D FFT λ_d of $\mathbf{d}_1$ on $N \times N$ points.
3. Compute the 2D FFT $\mathbf{Y}$ of data.
4. Build the frequency transfer matrix $\mathbf{g}_{RLS}$ similar to part 2.
5. Compute $\widehat{\mathbf{X}}$ as the product of $\mathbf{g}_{RLS}$ by $\mathbf{Y}$.
6. Reconstruct $\widehat{\mathbf{x}}$ by computing the inverse 2D FFT of $\widehat{\mathbf{X}}$.

Justify and comment on the choice of filter

$$\mathbf{d}_1 = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

as a regularization operator. How is the criterion $\mathcal{F}_\alpha(\mathbf{x})$ defined?

3.3 Numerical simulation

3.3.1 Direct problem

Load the image `cameraman.tif` into the matrix $\mathbf{x}$, of size 256×256 pixels. This is the image we will seek to restore later.

Filter the image `cameraman.tif` by the averaging filter :

$$\mathbf{h}_1 = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix},$$

(filtering by `conv2` and in 'same' mode) and add a white and Gaussian noise, so as to ensure an SNR of 30 dB.

Display image $\mathbf{x}$ and the blurry and noisy image $\mathbf{y}$.

Display the frequency response $H_1(\nu_x, \nu_y)$ of the PSF : identify both frequency axes, the location of the zero frequency, of the maximum frequencies, *etc.* Check that the filter is low pass.

3.3.2 Implementing deconvolution

Your program should implement deconvolution using the least squares minimization and regularized least squares.

Elaborate on the general appearance of the reconstructed image and on the preservation of edges between neighboring regions (for example between the coat of the photographer and the sky).

What is going on with the inverse filter and why?

With the regularized method, explain the results qualitatively obtained. What is the influence of the α parameter? Compare the results obtained with the expected results given the choice of the regularization operator.

4. Edge-preserving deconvolution method

To restore an image with clear contours, we seek to minimize a criterion of the type :

$$\mathcal{F}_\alpha(x) = \|y - h * x\|_F^2 + \alpha \sum_{p \sim q} \varphi(x_p - x_q) \quad (5)$$

where $\sim$ represents a neighborhood relationship between pixels. We consider a 4-neighborhood relation, and choose

$$\varphi(t) = \sqrt{t^2 + T^2} - T. \quad (6)$$

The goal is to implement the optimization of $\mathcal{F}_\alpha(x)$ by calling solver `fminunc`, and compare the results obtained with those of the quadratic regularization method.

It is recommended to proceed step by step, as detailed below.

4.1 Denoising and deconvolution of piecewise constant signals

We will start by testing the method in 1D case, for denoising of a piecewise constant signal. Once you have written the codes, you will have to modify the different stopping conditions of the algorithm to understand their influence on the obtained results. You also have to test the influence of the initial solution and the hyperparameters α and T on the results. Once your program has been tested, you can generate comparative results with the quadratic regularization method.

4.2 Deconvolution of small images

Now, the previous processing task is extended to the deconvolution of a small image, by adapting the 1D codes to image processing. Again, you will generate comparative results with the quadratic regularization method.